Butterbear

Campground Booking REST API

Technical handoff and presentation guide

Prepared for **Nemo and Aon**. Reviewed against commit `bf6eeb2`, dated 11 November 2026.

What this document gives you

A requirement map, source-file guide, database explanation, endpoint reference, email and weather flows, setup instructions, demonstration sequence, and answers to likely professor questions. Read the overview together, then study your assigned technical packages.

Project identity

Item	Current implementation
Repository	`https://github.com/chanichkorn/butterbear`, public
Local folder	`/Users/nemo/Documents/GitHub/butterbear`
Base API	`http://localhost:5005/api/v1`
Health check	`http://localhost:5005/health`
Backend	Node.js 22 or later, Express, MongoDB and Mongoose
Demo email	Nodemailer with Ethereal test inboxes
External data	Open-Meteo weather forecast API
Client	Postman collection in `postman/`

Implemented scope

Users can register, log in, log out, browse campgrounds, maintain favorites, and create a booking of one to three nights with a special request. Users can view, edit, and cancel their own bookings. Admins can manage any booking and also manage campground records.

Booking creation queues a confirmation and a reminder. Booking edits queue an update email and a replacement reminder. The reminder fetches the check-in forecast and includes it in the email.

Current completion boundary

The API and integrations have been exercised locally. Ethereal receives test messages and exposes browser previews. Actual inbox delivery requires a real SMTP account and authorized sender. The brief permits MongoDB Atlas or PostgreSQL, so configure Atlas for submission. The development setup uses a separate local MongoDB database on port 27019.

Butterbear has not been publicly deployed. A running Node process is required for reminders. The earlier VacQ AWS deployment belongs to a different project.

How to describe the work split

The implementation was prepared with AI assistance. Nemo/Aon labels below assign **review, verification, maintenance, and presentation responsibilities**. They define an equal allocation for understanding and presenting the AI-assisted implementation. Each presenter should claim the work they actually review, test, modify, and demonstrate.

1. Equal handoff responsibilities

Three complete technical packages per person, with equal presentation time

Package	Lead	Responsibility and proof
N1	Nemo	Authentication, session storage, logout revocation, input validation, and confidentiality. Explain `auth.js` and auth handlers in `app.js`. Demonstrate login and rejection after logout.
N2	Nemo	Campground browsing/admin management, favorites, unique membership, and pagination. Explain User/Campground fields in `models.js`. Demonstrate add, repeat add, list, and remove.
N3	Nemo	Booking creation, calendar-night validation, timezone conversion, and special requests. Explain `bookingDates()` and the create handler. Demonstrate three nights accepted and four rejected.
A1	Aon	Booking reads/edits/cancellation, owner/admin scope, revision checking, and notification visibility. Explain update/cancel handlers. Demonstrate another user's rejection and an admin edit.
A2	Aon	Confirmation/update emails, embedded jobs, claims, leases, retries, cancellation handling, and SMTP/Ethereal. Explain `mail.js` and `notifications.js`. Show email previews.
A3	Aon	Weather API, forecast validation/cache/fallback, reminder rescheduling, startup, and end-to-end verification. Explain `weather.js`, `server.js`, and test strategy. Show the weather reminder.

The allocation is 50/50 by complete responsibility packages and speaking time. Authentication and booking rules require different work from external integrations, so source-line counts are not a reliable measure of equality.

Suggested 12-minute presentation

Time	Speaker	Coverage
0:00–1:00	Nemo	Scope, architecture, original requirements
1:00–3:00	Nemo	Authentication and campground/favorite behavior
3:00–6:00	Nemo	Booking dates, special request, creation demo
6:00–8:00	Aon	Ownership/admin scope, edit and revision behavior
8:00–10:00	Aon	Job persistence, email worker, message previews
10:00–12:00	Aon	Weather reminder, verification, deployment boundary
Q&A	Both	Package lead answers first. Both know the booking lifecycle

Shared preparation

1. Each person opens the files in their packages and traces a request from input to database write and response.
2. Nemo creates the booking that Aon edits. Both keep the same saved BOOKING_ID.
3. Aon inspects email previews before cancellation. Cancellation prevents further worker claims.
4. Swap roles once in rehearsal. Both must explain why login alone does not authorize access to someone else's booking.

2. Requirement traceability

Map each requested behavior to its implementation

Original Project 2

ID	Implemented behavior	Main implementation
B1	Register with name, phone, email, password	Registration route, User
B2	Email/password login and session logout	`auth.js`, Session
B3	Browse campground name/address/phone and book 1–3 nights	Campground/create routes
B4	View own bookings	`bookingOwner()`
B5	Edit own booking	PATCH booking route
B6	Cancel own booking	DELETE booking route
B7	Admin views all active bookings	Role-aware read/list scope
B8	Admin edits any active future booking	Same edit handler, admin scope
B9	Admin cancels any active booking	Same cancel handler, admin scope

Butterbear additions

ID	Implemented behavior	Main implementation
A1	Add/remove own favorite campground	Favorite PUT/DELETE
A2	View own favorites	Favorite GET/populate
A3	Queue confirmation after successful booking	`notificationJobs()`
A4	Fetch external weather and include it in the reminder before check-in	Weather service and worker
A5	Queue email after successful booking edit	Revision and appended jobs
A6	Add/edit a special request	`specialRequest`, max 2,000 characters

A4 reflects the agreed extension. The reminder is retained and enriched with an external weather API. Weather is fetched near check-in, so a booking can be created beyond the forecast horizon.

Non-functional requirements

- **Authentication and confidentiality.** Password hashing, signed JWTs, stored sessions, role checks, and owner filters protect private bookings. Production needs HTTPS for tokens in transit.
- **Three-second response target.** The verified local user workflow's slowest request was weather at 916 ms. The admin workflow's slowest was 276 ms. These are observations, not a production latency guarantee or load-test result.
- **Postman usability.** The collection supplies user, cancellation/logout, and admin workflows. No frontend is required by the brief.
- **Database constraint.** The implementation supports MongoDB Atlas. Configure Atlas for submission because the current local development database differs from the stated Atlas/PostgreSQL constraint.

3. Architecture and source map

One Express API with a database-backed email worker

```
Postman -> Express middleware -> authentication / authorization
        -> validation -> MongoDB write -> JSON response

Background worker in the same Node process
  -> claim a due embedded notification
  -> fetch Open-Meteo for a reminder
  -> Nodemailer SMTP / Ethereal
  -> save status and preview URL
```

File	Responsibility
`src/server.js`	Validate configuration, connect DB, initialize indexes/mail, start worker/API, handle shutdown.
`src/app.js`	Routes, headers, rate limits, request size, owner filters, booking writes, centralized errors.
`src/models.js`	User, Session, Campground, Booking, notification schemas and indexes.
`src/auth.js`	JWT/session creation and verification, current user/session, admin guard, owner scope.
`src/validation.js`	Typed input checks, IDs, coordinates/timezone, dates, pagination, ApiError.
`src/config.js`	Required environment settings and production SMTP requirement.
`src/services/mail.js`	Transport factory, event snapshots, notification jobs, plain-text messages, previews.
`src/services/notifications.js`	Claims, leases, recovery, revision checks, forecast, retries, polling lifecycle.
`src/services/weather.js`	Provider URL, timeout, data validation, normalization, bounded cache.
`scripts/seed.js`	Sample campgrounds and optional admin from local environment settings.
`test/api.test.js`	Ten tests using isolated MongoDB and controlled weather/mail dependencies.
`postman/Butterbear.postman_collection.json`	Runnable examples, demo variables, assertions, setup notes.
`.env.example`	Safe template without real credentials.

Architecture rationale

Express and MongoDB match the earlier backend coursework. Small route handlers remain together in `app.js`. Reusable authentication, validation, and external-service logic have separate modules.

Jobs live inside the booking document. Creating a booking and its email work is one atomic MongoDB write, avoiding a booking saved without its job after a separate queue insert fails. No Redis or external job broker is needed.

Email delivery is asynchronous relative to HTTP. A five-second timer in the API process performs the work. This is an application worker, not a separately hosted cron scheduler. The database persists the schedule across process restarts.

4. Database model

Know the relationships before explaining the routes

Collection	Fields and meaning
Users	`name`, `phone`, lowercase unique `email`, hashed `password`, immutable `role`, `favorites[]` referencing Campground, timestamps. Password is excluded from ordinary queries and JSON output.
Sessions	UUID `_id`, user reference, `expiresAt`. A TTL index eventually removes expired sessions, while authentication explicitly checks expiry.
Campgrounds	Name, address, phone, latitude $[-90, 90]$, longitude $[-180, 180]$, IANA timezone, active flag, timestamps. Default timezone is `Asia/Bangkok`.
Bookings	Owner and campground references, date-only checkIn/checkOut, absolute checkInAt, specialRequest, revision, status, embedded notifications, timestamps.

Relationships and ownership

One user can own many bookings. One campground can appear in many bookings and favorite lists. Each booking owns its notification history. `populate('campground')` returns campground information for booking reads.

The owner is taken from authentication during creation. A request body cannot transfer ownership to another user. Registration always assigns the normal-user role.

Embedded notification schema

Fields	Purpose
`_id`, `type`, `revision`	Unique job, confirmation/update/reminder type, booking revision that created it.
`recipient`, `snapshot`	Owner email and saved campground name/address/coordinates/timezone, dates, special request.
`scheduledAt`, `nextAttemptAt`	Earliest delivery time and retry eligibility. Both must be due.
`status`, `attempts`	Delivery state and number of processing attempts.
`claimToken`, `leaseUntil`	Unique worker claim and its temporary expiration.
`sentAt`, `messageId`	Completion time and message identifier. Preview mode also sets sentAt for test transport completion.
`previewUrl`, `lastError`	Ethereal inspection link or generic delivery failure message.

Indexes and privacy

Unique email prevents duplicate accounts, including concurrent registration attempts. Booking indexes cover user/status and status/notification retry time. Session expiry uses a TTL index.

Notifications are excluded by default using `select:false`. Creation explicitly strips them from the public booking result. The notification endpoint returns a limited summary after owner/admin checks. Treat preview URLs as private links.

Logical deletion

Cancellation retains the booking with `status:'cancelled'`. Active lists and normal reads hide it. The owned notification endpoint can still inspect its history. Campground deactivation likewise retains its record with `active=false`.

The model does not allocate individual pitches or enforce campground capacity. Those concepts were not specified in the supplied requirements.

5. Authentication and security

Nemo leads. Both presenters must understand owner filters

Registration and login

Inputs are typed and validated. Email is trimmed and lowercased. Passwords require at least eight characters and at most 72 bytes, respecting bcrypt's input limit. Production password hashing uses bcrypt cost 12.

Login explicitly selects the stored hash and compares it with the submitted password. On success, `issueSession()` creates a UUID session ID, signs a JWT using HS256, and stores a Session. The token uses `sub` for the user ID and `jti` for the session ID. Default lifetime is one day.

```
Authorization: Bearer <JWT>
```

Protected-request flow

1. Require a Bearer header.
2. Verify JWT signature, allowed algorithm, and expiry.
3. Find the matching unexpired database Session.
4. Load the current User and attach it to `req.user`.
5. Check admin role or include the owner constraint in the database query.

JWT payloads are readable even though they are signed. Passwords and booking details are not stored in the payload. The current role is read from MongoDB rather than trusted from user input.

Logout is observable

Logout deletes the current Session. Reusing its JWT then fails even before its signature expiry. Another login session remains valid. The TTL index is cleanup rather than the only expiration check.

Confidentiality

`bookingOwner(req)` returns an owner constraint for a normal user and no additional owner restriction for an admin. Routes combine it with the booking ID. Another user's booking returns 404, reducing disclosure of whether an ID exists.

Public registration cannot create admins. The seed configuration creates an admin locally. Favorite operations always target the authenticated user's document.

Other controls

- Helmet headers and disabled Express identifying header.
- JSON limit of 20 KB; API limit of 300 requests per ten minutes per IP. Extra login limit of 30 per fifteen minutes.
- Typed email/string checks reject object/operator-style login payloads.
- Handlers select accepted body fields. They do not pass an arbitrary request object into the database.
- IDs must be 24 hexadecimal characters. Pagination and string lengths are bounded.
- Hashes/internal errors are excluded from ordinary output. Secrets remain in ignored environment files.

Local HTTP is for development. Deployment needs HTTPS. The rate limiter uses process-local memory and would need shared storage for coordinated limits across several instances.

> *We verify both the signed token and the stored session, then enforce booking ownership. Authentication identifies the caller. Authorization decides which records that caller can access.*

6. Account and campground endpoints

All listed paths are relative to /api/v1

Method	Path	Access and normal result
POST	`/auth/register`	Public, 201, user/token
POST	`/auth/login`	Public, 200, user/token
GET	`/auth/me`	Signed-in user, 200
POST	`/auth/logout`	Signed-in user, 200
GET	`/campgrounds`	Public, 200, active list
GET	`/campgrounds/:id`	Public, 200, active record
GET	`/campgrounds/:id/weather`	Public, 200, forecast
POST	`/campgrounds`	Admin, 201
PUT	`/campgrounds/:id`	Admin, 200, validated full input
DELETE	`/campgrounds/:id`	Admin, 200, deactivate
GET	`/favorites`	Signed-in user, 200, own list
PUT	`/favorites/:campgroundId`	Signed-in user, 200, add
DELETE	`/favorites/:campgroundId`	Signed-in user, 200, remove

Registration example

```
{
  "name": "Demo Camper",
  "phone": "0800000000",
  "email": "demo-camper@example.test",
  "password": "<a new demo password>"
}
```

The result contains success, token, and user data without the password hash. Login accepts email/password only. Do not use the placeholder password literally.

Campground behavior

Create and PUT require name, address, phone, numeric coordinates, and an optional valid IANA timezone. PUT validates the complete input, so include all required fields.

The list supports `?page=1&limit=25`, maximum limit 100 and maximum page 100,000. Results include count, total, page, and data. Ordering uses name then ID.

Deactivation returns 409 if the check finds active future bookings. It sets active false. This check and a concurrent booking insertion are separate operations, so it is not a cross-document transactional guarantee.

Favorites

Add uses `\protect\TU\textdollaraddToSet`, preventing duplicate memberships during concurrent requests. Remove uses `\protect\TU\textdollarpull` and succeeds when already absent.

Add requires an active campground. Removal only needs a valid ID, allowing removal after deactivation. Favorite listing populates active campgrounds only.

Forecast

```
GET /api/v1/campgrounds/<id>/weather?date=2026-11-12
```

Use a current supported date during the demo. Omission defaults to today in that campground’s timezone.

7. Booking endpoints and payloads

Nemo explains creation; Aon explains edits and admin scope

Method	Path	Normal behavior
POST	`/bookings`	201, create own booking/jobs
GET	`/bookings`	200, own active bookings. Admin sees all
GET	`/bookings/:id`	200, owner/admin
PATCH	`/bookings/:id`	200, edit and queue jobs
DELETE	`/bookings/:id`	200, cancel
GET	`/bookings/:id/notifications`	200, owner/admin summaries

Create example

```
{
  "campground": "<active campground ID>",
  "checkIn": "2026-11-12",
  "checkOut": "2026-11-15",
  "specialRequest": "A quiet shaded pitch near the restroom."
}
```

Replace example dates when they are past. This is a three-night stay. The API supplies the owner from authentication. Special request is optional, trimmed, and limited to 2,000 characters. It is included as plain text in emails.

A successful result includes `emailStatus:"queued"`. It proves the jobs were persisted, not that a recipient already received a message.

Edit example

```
PATCH /api/v1/bookings/<booking ID>
{
  "specialRequest": "Please reserve a shaded pitch."
}
```

Allowed edits are campground, checkIn, checkOut, and specialRequest. Missing values retain current fields. An empty string clears the request. Explicit null retains it because the handler uses nullish fallback.

The chosen campground must still be active. Dates are revalidated and an already-started booking cannot be edited. Admin edits address the email to the booking owner. Ownership cannot be changed through this payload.

Cancellation and notifications

DELETE marks cancelled and increments revision. Normal reads no longer return it. Repeating DELETE gives 404. No cancellation email is implemented because it was not required.

The notification endpoint can still inspect an owned cancelled booking. Its summaries include job ID/type/revision, state, schedule/completion, attempts, preview link, and generic error. It excludes recipient/snapshot/worker claim fields.

Status codes

Code	Meaning
400	Invalid input, ID, booking dates, or unsupported forecast date
401 / 403	Authentication failure / authenticated non-admin on an admin route
404	Missing, inactive, cancelled, or inaccessible record depending on route
409 / 429	Conflicting state or duplicate email / rate limit
500 / 503	Internal or external service failure. Generic service-unavailable message

8. Dates and booking revisions

Calendar rules, scheduling policy, and concurrent updates

One to three calendar nights

`bookingDates()` requires date strings shaped as YYYY-MM-DD. Luxon parses the dates in the campground timezone and rejects invalid dates. Check-out minus check-in must be an integer from one to three calendar days. This is a per-booking stay limit. The API does not cap the user's total number of bookings.

Calendar arithmetic avoids treating a daylight-saving day as exactly 24 hours. The automated test covers a three-night stay across a clock change.

Times and their meaning

Value	Calculation
checkIn/checkOut	Date-only strings describing the stay
checkInAt	Check-in date at 14:00 in the campground timezone, converted to an absolute Date
Normal reminder	Day before check-in at 09:00 in the campground timezone
Late booking/edit	If that reminder time passed, schedule it now while check-in remains in the future

For 12 November 2026 check-in in Bangkok, normal reminder time is 11 November at 09:00 and arrival is 12 November at 14:00. Creating the booking on 11 November at noon makes the reminder immediately due. This allows a real reminder demo without changing the machine clock.

The exact times were implementation choices because the requirement only says before check-in. Worker polling and SMTP add delivery delay. Once check-in has passed, an overdue reminder is skipped.

Optimistic revision check

Revision begins at 1. An edit reads revision r, then conditionally updates only the document that still has revision r. The atomic write sets the fields, writes $r + 1$, and appends update/reminder jobs for that new revision.

If two edits read the same revision, the first matching write wins and the second gets 409. If the second request reads after the first completed, it can update the new revision normally. This protects the read-then-write interval. The client does not submit an If-Match version.

Old reminders remain in history. Their job revision differs from the booking's current revision, so they are superseded. The worker checks again after weather lookup in case another edit occurred during the external request.

Snapshots

Each job stores campground name/address/coordinates/timezone, check-in/out, and the special request at event time. Later campground metadata edits do not rewrite old snapshots. Editing the booking creates a new snapshot and reminder.

Confirmation and update jobs describe their original events. Revision supersession specifically governs reminders. A late edit can therefore produce a second reminder for the new revision even if an earlier reminder was already sent.

> *The booking revision connects reminder delivery to the current booking details. We preserve old jobs as history but prevent an old-revision reminder from being selected as the current reminder.*

9. Email worker and reliability

Aon traces deliverNext() from claim to completion

Atomic persistence

Booking creation and its two notification jobs are saved in one document. Editing fields and appending new jobs uses one conditional update. This avoids separate booking/queue writes without requiring a multi-document MongoDB transaction.

```
pending -> processing -> sent       (real SMTP processing)
                      -> preview    (Ethereal test transport)
                      -> pending    (retry later)
                      -> failed     (fifth failed attempt)
                      -> superseded (no longer applicable)
processing with expired lease -> pending
```

One delivery attempt

1. Reset processing jobs with expired leases to pending.
2. Atomically find one active booking with a pending job whose schedule and nextAttemptAt are both due.
3. Mark it processing, attach a random claim token and a 60-second lease, and increment attempts.
4. Find that exact job in the returned document using the claim token.
5. Skip outdated or past-check-in reminders. Fetch weather for a current reminder.
6. Re-read booking status/revision after the external lookup. Supersede jobs that are no longer applicable.
7. Compose text and call the SMTP transport.
8. Update the job using its ID, claim token, and processing state so a stale worker cannot normally overwrite a replacement claim.

The timer ticks at startup and every five seconds. A running flag prevents overlapping ticks in one process. Each tick handles up to 20 jobs sequentially.

Failure and backoff

SMTP rejection or processing failure retains the booking. Delays after failed attempts 1–4 are 1, 2, 4, and 8 minutes. Attempt five makes the job failed. The generic lastError avoids exposing SMTP details.

The worker does not offer a public failed-job retry route. Investigate configuration first and perform deliberate maintenance if retrying a failed job.

Guarantees and limits

Normal concurrent workers atomically claim different jobs. A lease allows recovery after a crashed worker. The database preserves pending work across restarts, but a running Node process must return to deliver it.

The system uses at-least-once retry behavior. A crash after SMTP acceptance but before saving success can duplicate an email. The stable Message-ID contains booking/job IDs and helps identify that retry, but does not guarantee provider deduplication.

The final cancellation/revision check reduces races. It cannot recall a message already in flight or eliminate the small gap between checking and calling SMTP. SMTP acceptance also does not prove final inbox placement or that a person read the message.

10. External weather integration

Forecast lookup enriches the pre-check-in reminder

Provider request

```
GET https://api.open-meteo.com/v1/forecast
  ?latitude=<campground latitude>
  &longitude=<campground longitude>
  &daily=weather_code,temperature_2m_max,
         temperature_2m_min,precipitation_probability_max
  &timezone=<campground timezone>
  &start_date=<check-in date>
  &end_date=<check-in date>
```

The actual URL is encoded with URLSearchParams without line breaks. The public endpoint is used without an API key.

Only coordinates, timezone, and date are needed by Open-Meteo. The request does not include a user's name/email/password, booking ID, or special request.

Input and output checks

The date must be today through today plus 15 days in the campground timezone, a total range of 16 dates. Returned daily time must match the requested date. Weather code, minimum/maximum temperature, and rain probability must all be finite numbers.

The normalized result includes provider attribution, date/timezone, weather code, Celsius temperatures, and precipitation probability. The app reports the numeric weather code without mapping it to a written condition.

Timeout and cache

Fetch uses a two-second abort timeout. Bad HTTP status, network/JSON failure, or incomplete data becomes a 503 error. The HTTP error handler uses a generic service-unavailable message.

Successful values are cached for 15 minutes. The cache key combines coordinates, timezone, and date. At 500 entries, the oldest inserted key is removed before insertion. It is a process-local bounded insertion-order cache, not shared storage or a full LRU implementation.

Why delivery-time lookup matters

A booking can be months away even though the provider cannot forecast that far. The worker waits until the reminder is due, normally one day before arrival. That places the required forecast near the provider's supported range.

A weather outage does not suppress the reminder. It still sends with a forecast-unavailable note and keeps the booking confirmed. The public forecast route instead returns an error.

Message content example

```
Check-in forecast: 19 to 27 degrees C.
Rain probability: 40%.
Weather code: 3.
Source: Open-Meteo (https://open-meteo.com/).
```

These numbers illustrate formatting. The actual text uses a Celsius symbol and live/cached values. A controlled test verifies the 19–27 example. A separate live request confirmed provider connectivity.

> *The external API supplies weather. Email is its delivery channel. We fetch near check-in rather than pretending to have a forecast months in advance.*

11. Email transport and message content

Explain test delivery separately from real mailbox delivery

Operation	Generated work
Creation	Confirmation due now and reminder at the calculated time
Booking edit	Update due now and a new-revision reminder
Admin edit	Same behavior, addressed to the booking owner
Cancellation	No cancellation email. No further claims from cancelled bookings

Every message includes booking ID, campground name/address, check-in date/time/timezone, check-out date, and special request or None. Reminders add forecast/attribution or an unavailable-weather note. Messages are plain text, so special requests do not become HTML markup.

Ethereal demo mode

With `MAIL_TRANSPORT=ethereal`, Nodemailer creates a temporary test account and SMTP transport. Messages can be inspected but are not delivered to a real recipient's mailbox. The worker records status preview and a browser previewUrl.

Retrieve owned notification summaries and open that link. This proves generated content and test SMTP processing. Temporary-account/preview retention is controlled by the provider. Do not treat it as permanent message storage.

Real delivery configuration

```
MAIL_TRANSPORT=smtp
MAIL_FROM=Butterbear <an authorized sender>
SMTP_HOST=<provider SMTP hostname>
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=<provider account>
SMTP_PASSWORD=<provider credential>
```

Port 465 normally uses SMTP_SECURE=true. Port 587 normally negotiates STARTTLS without implicit TLS. Use the selected provider's settings.

Real SMTP calls transport.verify() before API startup. Connection timeout is ten seconds and socket timeout is fifteen seconds. Startup rejects missing required settings or failed verification.

Production configuration requires `MAIL_TRANSPORT=smtp`. A deployed instance also needs an authorized sending identity, provider/network access, HTTPS for the API, and a running backend. Keep credentials in environment settings.

Delivery state meanings

- **queued** is the API response after persistence.
- **processing** means a worker has claimed the job.
- **preview** means the test transport completed.
- **sent** means real transport success was recorded.
- **failed** means attempts were exhausted.
- **superseded** means the work no longer applies.

Neither sent nor preview proves a human read a message. In preview mode, sentAt is a test-processing timestamp.

12. Setup and operations

Existing local demo and fresh-checkout instructions

Existing project on Nemo's Mac

Check `http://localhost:5005/health`. If it succeeds, the API is already listening. Otherwise start it from the prepared folder.

```
cd /Users/nemo/Documents/GitHub/butterbear
npm start
```

The prepared ignored .env contains generated admin credentials, a JWT secret, and `mongodb://127.0.0.1:27019/butterbear`. Inspect locally for the admin demo and keep secrets out of slides. A second server on the same port gives an address-in-use error.

Fresh checkout

```
git clone https://github.com/chanichkorn/butterbear.git
cd butterbear
npm ci
cp .env.example .env
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# Put the generated value into JWT_SECRET.
# Set MONGO_URI and optional ADMIN_EMAIL / ADMIN_PASSWORD.
npm run seed
npm start
```

Node 22 or later is required. The repository is public, so Aon can clone it without a collaborator invitation.

The example URI assumes MongoDB on port 27017. The prepared local instance uses 27019. A fresh checkout does not automatically install/start persistent MongoDB. Use a normal local installation for development or Atlas. Configure Atlas for the submission database constraint.

Seeding

The seed upserts Khao Yai and Chiang Mai sample records by name without overwriting existing fields. With ADMIN_EMAIL/PASSWORD, it creates an absent admin. An address already owned by a normal user causes an error. It does not promote a normal account or reset an existing admin password.

Restart this Mac's local MongoDB if necessary

The development setup downloaded a MongoDB binary through the test dependency. From the project folder, this machine-specific command reopens the prepared persistent data directory.

```
node_modules/.cache/mongodb-memory-server/mongod-arm64-darwin-7.0.24 \
  --dbpath .local/mongodb --port 27019 --bind_ip 127.0.0.1 \
  --logpath .local/mongodb.log --fork
```

On another machine, use its MongoDB installation or Atlas. Do not start a second process on an occupied database port.

Lifecycle and configuration

Startup validates settings, connects MongoDB, initializes indexes, creates/verifies mail transport, starts the worker, and listens on port 5005. SIGINT/SIGTERM stops HTTP, waits for the active worker tick, closes mail, and disconnects the database.

Set TRUST_PROXY=1 only behind the single trusted reverse proxy for which it is intended. The earlier AWS deployment has not been reused for Butterbear.

13. Postman demonstration script

Nemo creates the booking; Aon demonstrates its lifecycle and email

The collection is `postman/Butterbear.postman_collection.json`. It is imported into Nemo's Postman workspace as **Butterbear Campground Booking**. URL defaults to `http://localhost:5005`. It uses collection variables, so no Postman environment is required.

Nemo's demonstration

1. Open User workflow. For a fresh run, clear EMAIL and PASSWORD. The folder script generates new demo values only when those variables are empty.
2. Run Register user, Log in, and My profile. Explain password hashing and the test script that stores TOKEN.
3. Run List campgrounds, which saves CAMPGROUND_ID. Open its details and explain the coordinates/timezone.
4. Add a favorite, inspect My favorites, repeat add to show uniqueness, then remove it.
5. Create the booking with a special request. The collection generates tomorrow as CHECK_IN and four days from today as CHECK_OUT, a three-night stay in Bangkok.
6. Show 201, saved BOOKING_ID, and emailStatus queued. Hand this booking to Aon.

Aon's demonstration

1. Read the same booking and edit the special request. Show the increased revision and queued update.
2. Refresh notification summaries after at least five seconds; SMTP may add more delay.
3. Open confirmation/update preview URLs and inspect the special request.
4. Open the current-revision reminder and identify temperature/rain values plus Open-Meteo attribution.
5. Explain why tomorrow's booking after 09:00 today gets an immediate reminder. Before 09:00, it is scheduled later today. A valid same-day booking before 14:00 is another demonstration option.
6. Set ADMIN_EMAIL/PASSWORD from local seed configuration. Admin login changes TOKEN. Show admin reads/edits/cancellation with the test booking ID.
7. Cancel only after inspecting emails. Finish with logout and rejection of the old session token.

Runner selection

Run **User workflow** alone to reproduce 14 requests and 16 assertions. Admin examples need credential variables and an existing test booking. Running all folders blindly can cancel before email inspection and fail admin login with empty credentials.

The collection intentionally has no wait for SMTP completion. A passed Runner proves the synchronous API flow. Refresh notification summaries separately to prove asynchronous email behavior.

Repeated-run pitfalls

- Retained EMAIL causes a second Register to return 409. Clear demo credentials for a new account or skip Register and log in.
- Missing/stale booking IDs require a fresh booking.
- Another user's token correctly returns 404 for this booking.
- Admin login replaces TOKEN. Log in as the demo user again before describing an ordinary-user operation.
- Postman may store recognized credentials in Vault. Keep the demo on the local app and ensure referenced values are available. This project does not configure a cloud Monitor.

14. Verification and failure demonstrations

Know what the tests prove and what remains environment-dependent

Ten automated tests

No.	Coverage in `test/api.test.js`
1	Reject operator-style login input, duplicate normalized email, anonymous creation, and non-admin campground creation.
2	Concurrent favorite additions remain unique. Lists are user-specific. Repeated removal succeeds.
3	Calendar-night rules cover daylight saving, invalid dates, zero nights, and four nights.
4	Creation queues two jobs and stores the request. Another user cannot read/edit/cancel/inspect the booking.
5	Admin edit creates a revision and addresses the update to the owner.
6	Concurrent worker calls normally claim each due confirmation/update once.
7	Current reminder includes controlled weather. Obsolete reminder is skipped.
8	SMTP failure persists a retry and preserves the booking. Cancellation prevents a later claim.
9	Weather URL/response checks, unsupported dates, and network failure work with a controlled fetch dependency.
10	Logout invalidates one session while a second login remains usable.

Run `npmtest`. Tests start an isolated temporary MongoDB and shut it down. They do not touch demo data or rely on real inbox delivery. Mail stream/stub transports and controlled weather responses make failure cases reproducible.

Live implementation-time checks

- User workflow passed 14 requests and 16 assertions.
- Admin workflow passed seven requests and seven assertions.
- Total live checks were 23 assertions across 21 requests.
- Open-Meteo returned a real forecast.
- A separate Ethereal SMTP test was accepted.
- The running API produced confirmation, update, and current-reminder previews. The old reminder was superseded.

These are recorded implementation-time results. IDs, weather, timestamps, and preview links change. Opening this document does not rerun the tests.

Useful negative demonstrations

1. Four-night booking returns 400.
2. Normal-user token on an admin-only campground route returns 403.
3. A second user’s attempt to read the first user’s booking returns 404.
4. After logout, the same JWT on /auth/me returns 401.
5. Weather 30 days ahead returns 400, while a distant booking can still be created because reminder weather is fetched later.

Expected 4xx responses demonstrate rules. A stopped database/API, unavailable SMTP provider, or expired demo session is an operational issue and should be diagnosed separately.

15. Professor questions and answers

Use these as rehearsal prompts rather than a memorized script

Why JWT plus database sessions?

JWT provides verifiable identity/expiry. A stored session enables immediate logout for one token without invalidating every other session. Every protected request checks both.

Why embed email jobs?

The booking and its required email work commit together. A separate queue insert could fail after booking success. Embedding fits the small project without an additional broker. Notification history growth is the trade-off.

Why respond before sending email?

SMTP latency and failure are outside the API's control. The route first persists the work and returns queued. The worker delivers/retries independently.

Can two campers book the same campground dates?

Yes. The requirements specify a maximum stay, but do not define pitch inventory or campground capacity. This implementation has no inventory locking, payment, or availability allocation.

What happens when a booking is edited?

The update validates new input, conditionally increments revision, and appends update/reminder jobs. Old reminder revisions become obsolete and are rechecked before delivery.

What happens after a restart?

MongoDB retains pending jobs. A new worker reads due work and reclaims expired leases. A crash after SMTP acceptance can cause a duplicate, so exactly-once email delivery is not promised.

What if weather is unavailable?

The public weather route returns a service error. The reminder still sends with a forecast-unavailable note. The booking remains confirmed.

Did a real customer receive email?

The demo verifies Ethereal SMTP processing and browser previews. Real mailbox delivery requires production SMTP credentials and an authorized sender. Do not describe previews as proof of customer inbox delivery.

Is the three-second target guaranteed?

Measured local requests were below three seconds and weather uses a two-second timeout. There is no production load test or hard end-to-end deadline. Database/network/load conditions still matter.

How can a user become an admin?

Only the seeded local configuration establishes an admin in this implementation. Public registration forces user role, and routes check the stored role.

What did each presenter contribute?

Explain the three assigned review/demo packages, the tests each person verified, and any modifications each actually made. Describe AI assistance according to course rules. The proposed split is a learning and maintenance allocation, not a fabricated contribution record.

16. Boundaries and final checklist

Keep presentation claims aligned with v1

Known implementation boundaries

- Atlas configuration remains necessary for literal compliance with the brief's Atlas/PostgreSQL database constraint. The current demo database is local MongoDB.
- Real email requires SMTP settings. Ethereal provides test previews.
- There is no public Butterbear deployment. A running backend is needed for reminders.
- Retries can duplicate an accepted message after a crash. In-flight emails cannot be recalled by concurrent edits/cancellation.
- Cancellation prevents new job claims. Historical pending jobs can remain in the document. Some non-reminder jobs can still appear pending in its summary.
- There is no public failed-job retry endpoint, bounce tracking, delivery webhook, capacity/pitch management, payment, or frontend.
- Campground deactivation and booking insertion have a race window because they are separate writes.
- Rate limits/cache are process-local. Embedded notification histories and favorite lists are not bounded or paginated like the main list endpoints.
- The health route checks that HTTP responds. It is not an ongoing database/SMTP readiness check.

Before presentation

1. Both can access the public repository and open their assigned source files.
2. Database/API are reachable and /health succeeds.
3. Postman URL is port 5005. Sample campground records exist.
4. Demo credentials/IDs are current. Secrets are kept out of slides.
5. Nemo explains bcrypt, session logout, favorite uniqueness, and calendar-night validation.
6. Aon explains revisions, worker claims/leases, retries, forecast lookup, and Ethereal.
7. Reminder timing is checked. Email previews are inspected before cancellation.
8. Both can demonstrate an authorization failure and explain the complete booking lifecycle.
9. The team describes its actual review, verification, and presentation work accurately.

Rebuilding this handoff

Compile this standalone source with XeLaTeX, LuaLaTeX, or `tectonicButterbear-Handoff.tex`. No external images are needed. The source uses Times New Roman/Menlo when installed and falls back to TeX Gyre Termes/Latin Modern Mono.

Source references

The code at `https://github.com/chanichkorn/butterbear/tree/bf6eeb2` is the implementation authority. Start at app.js, then inspect auth.js, models.js, validation.js and services. Recheck this handoff after those files change.

The original brief is *SW Dev Practice Project Listings*, Project 2 on page 6, with non-functional requirements/constraints on pages 2–3. Butterbear additions and the weather-in-email extension were provided in the project conversation.

- Forecast API. `https://open-meteo.com/en/docs`
- SMTP transport. `https://nodemailer.com/smtp`
- Ethereal testing. `https://nodemailer.com/guides/testing-with-ethereal`